

Data Structure (week 15)

資料結構補充

2026/6/1

靜態網站的 coding 語言與 python 不同

- 補充 1: HTML / CSS / JS 中 JS 全名是 JavaScript 為網站主要程式語言
 - JavaScript 對應到原本我們在學的 Python, 因此要做翻譯 (轉譯, 不同語言轉換)
 - HTML / CSS 不是程式語言, 只是架構和美觀用的存在 (所以很多人說 ...)
- 補充 2 : 靜態網站資料來源
 - 寫死在 JS 中, 就如同寫死在 Python 中一樣 (最為簡單)
 - 串連 google sheet, 可透過不同分頁、不同 column 去管理
 - 建議只開「所有人讀取權限」, 因為是公開的任何人都會知道該網址
 - 讓 AI 幫你做串連, 尤其對 Antigravity 來說是自家生態系很簡單

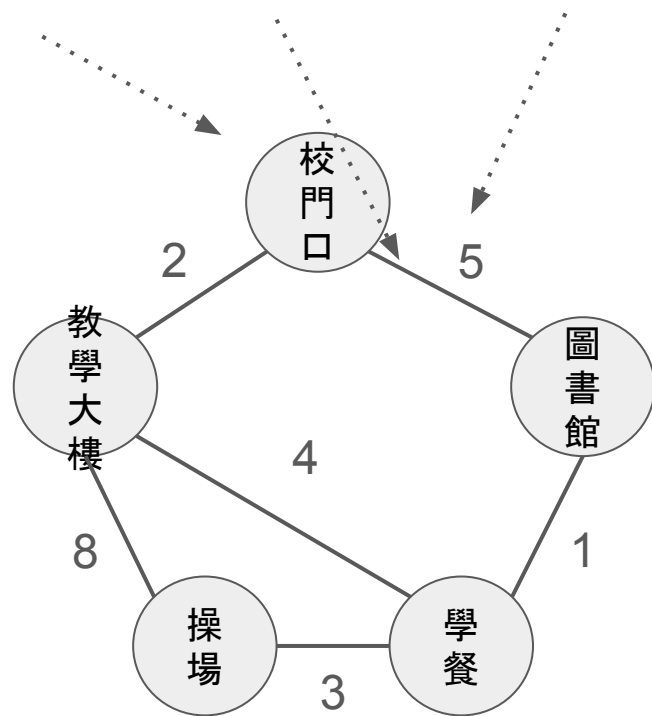
Graph 圖結構

- 目的：
 - 現實世界資料不一定是一直線 (Array 等) 或 Tree 結構 (單向往下走), 而是四通八達「網絡結構」
- 例子：
 - 北科大校園地圖
 - 節點 (Node) : 校門口、圖書館、教室、宿舍等
 - 邊 (Edge) : 連接兩個地點的走廊或馬路
 - 權重 (Weight) : 走這條路要付出的「成本 (代價)」(距離、時間、被雨淋的機率等)
- 實作方式：
 - 透過 Hash Map 但是用不同的組合方式

Graph 圖結構：實際案例

- 實作方式：
 - 透過 Hash Map 但是用不同的組合方式
 - campus_map = {
 - '校門口': {'圖書館': 5, '教學大樓': 2},
 - '教學大樓': {'校門口': 2, '學餐': 4, '操場': 8},
 - '圖書館': {'校門口': 5, '學餐': 1},
 - '學餐': {'教學大樓': 4, '圖書館': 1, '操場': 3},
 - '操場': {'教學大樓': 8, '學餐': 3}
 - }
- campus_map['教學大樓']['學餐'] 就能知道從 '教學大樓' 走向 '學餐' 的成本是多少 (access)。
- 一個 dict 代表一張 graph, 但 cost 只能訂一個意義。因次可以做多張 graph 對不同意義的 cost 做計算。

節點 (Node) 邊 (Edge) 權重 (Weight)

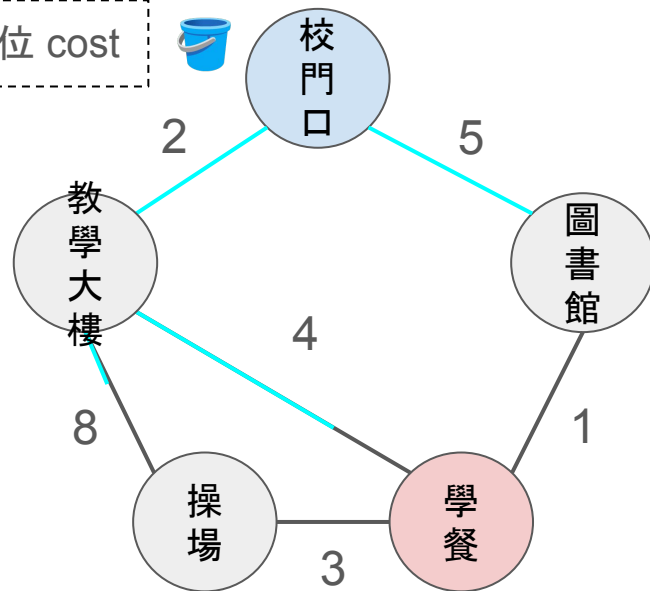


Dijkstra 演算法概要 (最穩健的尋路法)

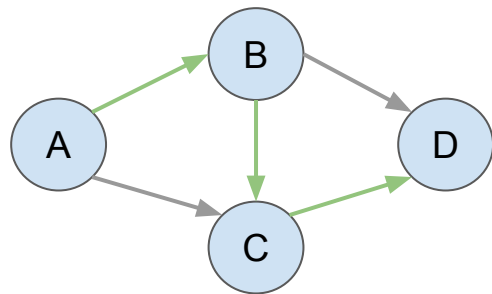
- 想像：

- 在「校門口」倒下一桶水，水會順著路往四面八方流。水流速度一樣的情況下，水最先流到的地方，就是最短路徑。
- 建立在 Graph 上的演算法

假設每秒流速 1 單位 cost



Dijkstra 演算法概要 (最穩健的尋路法)



- 想像：

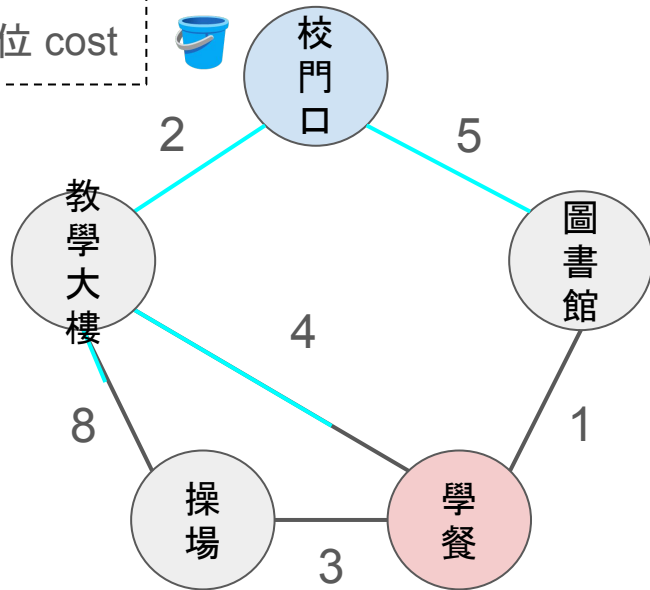
- 在「校門口」倒下一桶水，水會順著路往四面八方流。水流速度一樣的情況下，水最先流到的地方，就是最短路徑。
- 建立在 Graph 上的演算法

假設每秒流速 1 單位 cost



- 靜態網站 JS 實作建議：

- npm install **dijkstrajs**
- const graph = {
- 'A': { 'B': 1, 'C': 4 },
- 'B': { 'C': 2, 'D': 5 },
- 'C': { 'D': 1 },
- 'D': {} // (補充) 圖可以是單向，不一定要雙向
- };
- const path = dijkstra.find_path(graph, 'A', 'D'); // 見右上圖



Vibe Coding 基礎

AGENTS.md + PRD.md + SKILL



AGENTS.md

- 每個 Vibe Coding 內初始化時「**一定要有**」這份文件作為長期記憶 (or 重要資訊) , 不同 AI Coding 有不同名稱 e.g. [CLAUDE.md](#)
- 它會告訴 AI :
 - 這是什麼專案 + 執行環境 ?
 - 要用什麼技術 ?
 - 資料從哪裡來 ?
 - 哪些事情不能做 ?
 - 畫面風格如何設計 ?
 - 主邏輯要怎麼計算 ?
 - 常見的錯誤怎麼處理 ?

有後台、資料庫的這邊都
要定義清楚完整架構

AGENTS.md - Step 1: 先寫概要

AGENTS.md

Project Overview

本專案是「SafeRing: 校園災害韌性通報與避難路徑」的前端展示 Demo。

目標是做出一個可以在瀏覽器中打開的靜態網站, 展示:

- 校園地圖節點
- 災害發生位置
- 危險區域
- 安全避難路線
- 資料結構如何被使用

本專案主要用於課堂展示與期末報告, 不是實際災害系統。

AGENTS.md - Step 2: 規範 Tech Stack

Tech Stack

請只使用：

- HTML
- CSS
- JavaScript
- 不使用後端
- 不使用資料庫
- 不使用登入系統
- 不使用 npm / React / Vue 等框架

最終作品必須可以直接用 `index.html` 開啟。

File Structure

請維持以下檔案結構：

```
|— index.html
|— style.css
|— script.js
|— data/  # 任何其他資料都放置於此
|   |— campusGraph.js
|   |— disasterScenario.js
|— AGENTS.md
|— PRD.md
|— USER\_STORY.md
|— README.md
```

AGENTS.md - Step 3: Must Have + Do Not Do

Must Have

- 校園地圖或節點圖
- 災害點
- 危險區域
- 避難路線
- 使用 BFS 或 Graph 的說明
- 適合簡報展示的畫面

Do Not Do

- 不要做太複雜的系統
- 不要加入真實 API
- 不要重寫全部程式
- 不要做需要安裝才能跑的功能



AGENTS.md - Step 4: 驗收標準 (1)

驗收標準 (重要)

1. 基本執行

- 使用者可以直接打開 `index.html` 看到完整畫面
- 不需要安裝額外套件
- 不需要登入
- 不需要後端伺服器
- 不需要 API key
- 頁面載入後不能出現明顯錯誤畫面
- 瀏覽器 console 不應該有會影響功能的 JavaScript error

2. Demo 流程

使用者必須能完成一個完整展示流程：

1. 進入首頁。
2. 選擇或看到一個明確的情境。
3. 操作主要功能。
4. 系統產生明確結果。
5. 結果可以被截圖或口頭說明。
6. 整個流程可以在 3 分鐘內展示完。

如果是互動式作品，至少要有一條「保證可成功展示」的 demo path



AGENTS.md - Step 4: 驗收標準 (2)

3. 核心功能

- 至少完成 MVP 中定義的核心功能。
- 核心功能不能只是靜態文字描述，必須有可操作或可觀察的結果。
- 每個按鈕或主要互動元件都應該有明確作用。
- 不應存在點了沒有反應、但看起來像主要功能的按鈕。
- 如果功能尚未完成，請不要放在主要畫面上誤導使用者。

完成時必須能回答：

- 使用了哪一種資料結構？
- 資料是怎麼存的？
- 為什麼這裡適合用這種資料結構？
- 這個資料結構如何影響功能運作？

4. 檔案與程式碼

- 主要檔案命名清楚，例如 `index.html`，`style.css`，`script.js`。
- 不要把大量 JavaScript 全部塞在 HTML 裡。
- 不要留下大量沒用到的程式碼。
- 不要留下測試用假按鈕、假頁面或無意義文字。
- 函式名稱應該能看出用途。
- 如果有資料檔，資料結構要清楚，不要全部寫成難以閱讀的一大坨字串。



AGENTS.md - Step 4: 驗收標準 (3)

5. 穩定性

- 重新整理頁面後仍可以正常使用。
- 操作順序稍微不同時，不應直接壞掉。
- 沒有選資料就按按鈕時，系統應該給出提示，而不是報錯。
- 如果輸入錯誤，畫面應該有清楚回饋。
- Demo 前應至少完整跑過3 次主要流程。

6. 範圍控制

以下內容不是必要項目，不應該影響 MVP 完成：

- 登入系統
- 後端資料庫
- 真實金流
- 真實通知系統
- 真實 GPS
- 複雜 AI 模型
- 大量頁面
- 過度精緻但無法操作的動畫



AGENTS.md - Step 4: 驗收標準 (4)

10. 最終完成定義

當以下條件都達成時，才算完成：

- 可以正常開啟作品。
- 可以完成主要 demo 流程。
- 有清楚的輸入、處理、輸出。
- 至少一個資料結構有被實際用在功能中。
- 畫面適合展示。
- 團隊能說明作品邏輯。
- 沒有明顯會讓 demo 中斷的錯誤。

補充：以上驗收過程建議使用 browser use 功能 (/browser) 做測試

小組任務

- Step 1. 小組做簡單的討論 (非最終版), 大家一起 / 個別寫 User Story, 經討後留下最精萃的 User Story。
- Step 2. 第一步完成 User Story 後, 大家一起收斂成 AGENTS.md
- [檢核點] 完成後找我驗證, 會檢查 [USER_STORY.md](#) 和 [AGENTS.md](#)
- Step 3.
 - **全組** 收斂 USER_STORY.md 和 AGENTS.md 做出 (初步) [PRD.md](#) (具體要做哪些)
 - 「**設計**」根據需求設計出一版, 使用 Google Stitch / Figma / Canva 擅長的工具皆可
 - 「**軟體**」根據 PRD.md 進一步細化到各模組怎麼運作(要盡可能有詳細邏輯)
 - 「**架構**」用 draw.io 畫出一張 Data Flow Diagram (DFD 圖), 參考上一周內容
 - 「**PM**」同步追蹤 3 位組員進度, 機動參與了解主概念, 同時規畫產品開發時辰 (分階段即可)
- [檢核點] 全組完成後找我驗證, 會個別檢查每位進度 (善用 AI & Google)

以「校園災害韌性通報與避難路徑」為例子

- 架構圖要注意什麼？

- 先畫角色, 再畫系統

- **左邊**放使用者: 學生、老師、管理員
 - **中間**放網站系統: index.html、地圖 UI、警報模組、路徑演算法。
 - **右邊**放資料來源: 校園地圖資料、災害事件、集合點、模擬感測器資料。

- 每個模組都要寫「輸入 / 處理 / 輸出」

- 要寫清楚: 輸入是災害位置與目前位置, 處理是避開危險節點, 輸出是最佳避難路線。

- 箭頭要代表資料流, 不是裝飾

- e.g. 災害事件 -> 危險區域標記 -> 重新計算路線

- 要讓軟體同學看得出檔案怎麼拆

- index.html: 畫面骨架
 - style.css: 地圖與警報視覺
 - script.js: 事件、演算法、動畫
 - data/campusGraph.js: 節點與邊
 - data/disasterScenario.js: 災害情境 mock data

校園災害韌性通報與避難路徑（簡單架構圖範例）



專案檔案結構（前端靜態網站）

 index.html

 style.css

 script.js

 data/campusGraph.js

 data/disasterScenario.js

PM 隊長補充：

- 時間規劃，負責讓小組可以「準時做出作品」
 - 帶頭討論與擬定題目
 - 定使用者
 - 定 MVP
- 根據以上各寫出一份 PRD 和 User Story (功能優先順序)
- PM 重要性：
 - PM 的任務不是叫大家做事，而是把模糊的想法變成可以完成的任務。
 - 如果大家一直討論很大的功能，PM 要負責砍成 MVP (最小可行性產品)。
 - 如果 demo 做不完，PM 要決定先保留哪一個最能展示價值的流程。
 - 如果報告講不清楚，PM 要重排故事線。

Product Requirements Document (PRD)

- 產品是做什么？

- 我們要做一個校園災害避難輔助網站。當校園發生火災、地震或其他緊急事件時，使用者可以在網頁上選擇自己的位置與災害位置，系統會顯示建議避難路線、集合點，以及管理員發布的警報訊息。

- 解決誰的問題？

- 主要解決北科大學生、老師與校園管理人員的問題。學生和老師在不熟悉建築出口或災害位置時，可能不知道要往哪裡走才安全；管理員則需要一個簡單方式把警報與避難資訊快速傳達給校園內的人

- 為什麼值得做？

- 校園建築多、動線複雜，災害發生時如果只靠口頭通知或廣播，學生不一定能馬上知道安全路線。SafeRing 可以把校園地圖、危險區域、避難路線和通報流程視覺化，讓使用者更快理解「現在應該去哪裡」。

- MVP 包含哪些功能？

- 顯示簡化版校園地圖節點、使用者可以選擇目前位置、管理員可以選擇災害發生位置、系統會把災害點附近標成危險區域、系統用 BFS 或 Dijkstra 計算避開危險區域的避難路線、畫面用動畫顯示從目前位置到集合點的路徑、顯示一則警報訊息，例如「綜科館 3 樓發生火警，請避開 A 區」

User Story

- 格式

- 作為一位_____, 我想要_____, 因為_____。
- 重點不是寫功能名稱, 而是寫清楚: 誰會用? 他想完成什麼事? 為什麼這件事重要?
- User Story 重要性在於, 它把專題從「我們想做很多功能」拉回「我們正在幫誰解決什麼問題」。

- User Story 1: 學生避難

- 作為一位北科大學生, 我想要在災害發生時輸入我目前的位置,
- 並看到一條避開危險區域的避難路線, 因為我可能不熟悉每棟樓的出口與安全集合點。

- User Story 2: 老師確認學生集合點

- 作為一位授課老師, 我想要知道最近的安全集合點在哪裡,
- 並確認學生應該往哪個方向移動, 因為災害發生時我需要協助學生保持秩序並快速疏散。

- User Story 3: 管理員發布警報

- 作為一位校園管理員, 我想要快速選擇災害類型與發生位置,
- 並發布一則警報訊息, 因為我需要讓校園內的人立刻知道哪些區域要避開。

SKILL

1. Vibe Coding Skill 是什麼？



用自然語言描述需求，
和 AI 一起完成程式、作品或流程。



像是在當「導演」，
不是先學會每一行程式。



說清楚你想做什麼



把大問題拆成小步驟



請 AI 幫你產生、修改、解釋



自己檢查結果是否合理



想法



用中文
下指令



AI 產生
程式/內容



測試與修正



核心能力：邏輯、表達、判斷、不只是寫 code。

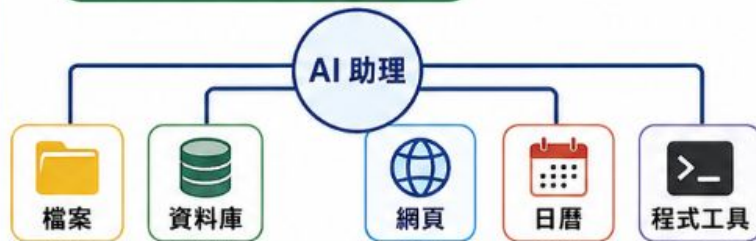
2. MCP 是什麼？

MCP = Model Context Protocol

讓 AI 可以安全地連接外部工具與資料的共通規則。



像 AI 世界的
「USB-C / 插座」。



- ✓ 讓 AI 讀取需要的資訊
- ✓ 讓 AI 使用工具幫你做事
- ✓ 不同工具可以用同一種連接方式



重點：有權限控制，不能隨便亂連。

一句話總結



Vibe Coding Skill：
學會跟 AI 合作解決問題。



MCP：
讓 AI 能接上工具與資料。



SKILL 安裝

- 直接給網址讓 Antigravity 2.0 安裝，以專案為主不要 global (自選)
- 前端設計 SKILL
 - 一般性：UI/UX Pro Max ([連結](#))
 - 進階：frontend-ui-engineering ([連結](#))
- 其他 SKILL 搜尋
 - <https://www.skills.sh/>
- 也能自己課製化更好

